

Stage 3 Report

Names: Petey, Kerry, Omid, Jahleel

Date: 4/25/2026

Stage 3 Code Documentation & Pipeline

For Stage 3, we integrated the feature extraction and classification pipeline into a complete supervised learning system. The purpose of this stage was to move beyond the unsupervised clustering and distance-based matching from Stage 2 and evaluate whether supervised classification could improve performance for image condition classification and identity re-identification. The main script used for this stage was `stage3.py`. This script imports the ArcFace and VGG feature extractors from `stage2.py`, loads the Gallery Set and Probe Set, extracts usable embeddings from the images, trains classification models, evaluates them on held-out test sets, and saves performance outputs such as confusion matrices and failure-case summaries. The pipeline uses the Gallery Set as the reference identity dataset and the Probe Set as the dataset containing altered images with conditions such as blur, face covering, and facial orientation changes.

Feature Extraction

The pipeline uses two pretrained feature extractors: ArcFace and VGG19. ArcFace produces a compact face embedding designed for identity matching, meaning images of the same person should ideally be close together in feature space. VGG19 features are also extracted and used as a second feature representation, especially for the supervised re-identification experiment. The feature extraction functions are inherited from the Stage 2 code, so Stage 3 builds directly on the feature extraction pipeline that was already developed earlier in the project. For condition and

severity classification, the script can use ArcFace features, VGG features, or a combined ArcFace + VGG feature vector. This is controlled using the command-line argument:

```
python Scripts/stage3.py --condition-feature-set both
```

Using ‘both’ combines the ArcFace and VGG embeddings into one feature vector for classification.

Condition and Severity Classification

The first supervised task was to classify the image condition and severity level. The script reads images from the Probe Set and assigns each image a condition label based on the folder structure and filename. The conditions include categories such as blur, face covering, and facial orientation. The script also assigns a severity label, such as slight, moderate, or severe, depending on the filename.

Three classification targets are evaluated:

- 1) condition
- 2) severity
- 3) condition + severity

The model used for this task is logistic regression. Before training, the features are standardized using StandardScaler, and then a multiclass logistic regression model is trained. The script uses a stratified train-test split so that the held-out test set keeps a similar distribution of

condition and severity labels. This is important because the test set should represent the same range of image conditions as the training set. The model reports training accuracy, test accuracy, and the train-test accuracy gap. This helps determine whether the model is generalizing well or overfitting. The script also prints a classification report and saves confusion matrices for each prediction task. In addition, misclassified examples are printed so we can examine failure cases and identify which conditions were most commonly confused.

VGG Re-Identification

The second supervised task was identity re-identification using VGG features. This was included because the Stage 2 results showed that vanilla VGG features were not always strong enough for distance-based face identification. To test whether supervised learning improved the VGG-based identity prediction, the script compares two methods:

1. Nearest-centroid baseline using VGG features
2. Logistic regression classifier using VGG features

The nearest-centroid method acts as the baseline. In this method, each identity is represented by the average location of its VGG feature vectors in feature space, and a probe image is assigned to the nearest identity centroid. This is similar to the clustering-style method used earlier, where images from the same person are expected to cluster near each other.

The supervised method trains a logistic regression classifier on top of the VGG features. The training set includes all gallery images plus a stratified split of known probe images. The remaining known probe images are held out for testing. This allows us to compare whether supervised classification improves identity prediction compared to the simpler nearest-centroid

baseline. The script reports the baseline train accuracy, baseline test accuracy, supervised train accuracy, supervised test accuracy, and train-test gap. It also prints a classification report, confusion matrix, and example failure cases for the supervised VGG re-identification model.

Failure Case Analysis

Failure cases are analyzed by comparing the true label and predicted label for each misclassified image. The script prints the most common true-to-predicted confusions and provides example file paths for misclassified images. These examples are useful because they allow us to inspect whether the model struggled due to image quality, face covering, blur, pose change, or similarity between identities. This failure-case analysis is important because accuracy alone does not fully explain model performance. For example, if the model performs well on slight blur but poorly on severe face covering, then the issue is likely related to missing facial information rather than the classifier itself. Similarly, if the train accuracy is much higher than test accuracy, this indicates that the model may be overfitting to the training images and not generalizing well to new probe images.

Output Files

The script saves output files in:

Scripts/stage3_outputs

The saved outputs include confusion matrices for the condition/severity models and the VGG re-identification models. These plots provide a visual summary of which labels were classified correctly and which labels were confused.

Running the Code

To run the Stage 3 pipeline, the user should run the following command from the project root or from the Scripts folder:

```
python Scripts/stage3.py
```

The default setting uses both ArcFace and VGG features for condition and severity classification.

To explicitly run the combined feature version, use:

```
python Scripts/stage3.py --condition-feature-set both
```

To run only ArcFace features:

```
python Scripts/stage3.py --condition-feature-set arcface
```

To run only VGG features:

```
python Scripts/stage3.py --condition-feature-set vgg
```

The script automatically loads the default Gallery Set and Probe Set paths from stage2.py. If a TA needs to use different folder locations, the variables near the top of stage3.py can be changed:

```
GALLERY_DIR_OVERRIDE = None
```

```
PROBE_DIR_OVERRIDE = None
```

These can be replaced with the correct local paths if needed. When the script starts, it prints the Gallery Set and Probe Set directories being used, which helps confirm that the correct data folders are loaded.

Summary

Overall, the Stage 3 pipeline uses ArcFace and VGG features as inputs to supervised classification models. For condition and severity classification, logistic regression is trained on extracted image features and evaluated using held-out test data. For identity re-identification, VGG features are evaluated using both a nearest-centroid baseline and a supervised logistic regression classifier. The pipeline reports train accuracy, test accuracy, train-test gap, classification reports, confusion matrices, and failure cases. When we tested our data, we got a percent of 84%, exceeding the recommendation by the rubric. This makes the system reproducible and allows the TA to evaluate the model on held-out test sets.

For testing day, our main objective was to use the feature vectors we were given and apply our pipeline to identify the unknown individuals as accurately as possible. To prepare for this, we took photos of the students in the class, kept track of each person's identity, and uploaded those images into our dataset so we would have a structured set of known faces to compare against. We then used those collected images to build our recognition pipeline and match the unknown embeddings to the most likely classmates. In this way, testing day acted as a practical evaluation of whether our system could use the stored class data to make accurate identity predictions under the challenge setting.

Final Predictions:

- Identity 1: Aaron Park
- Identity 2: Micheal Hunter
- Identity 3: Sanaya Kapoor
- Identity 4: Eli Boyden
- Identity 5: Ally O'Shea